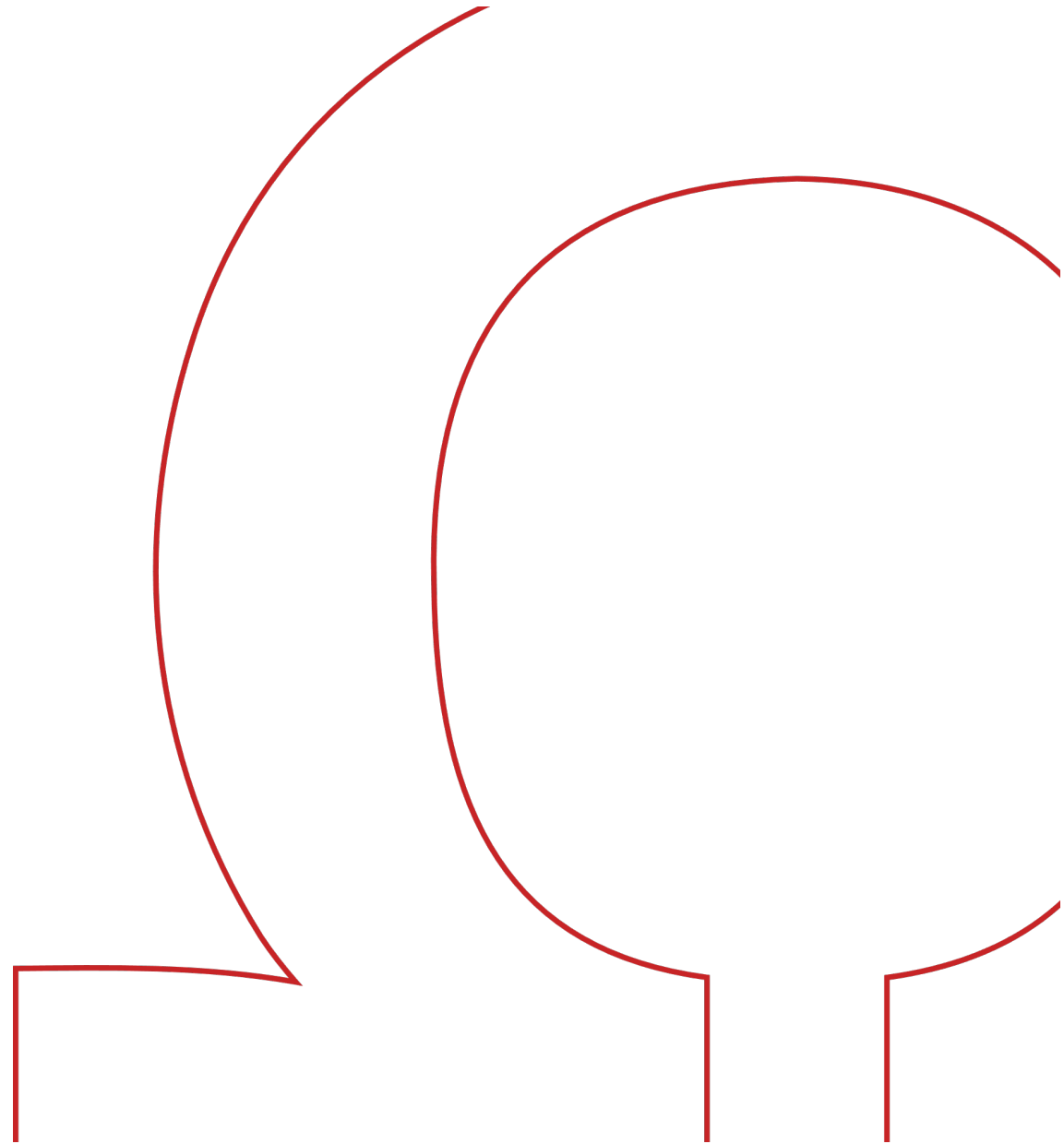


Prog III / A

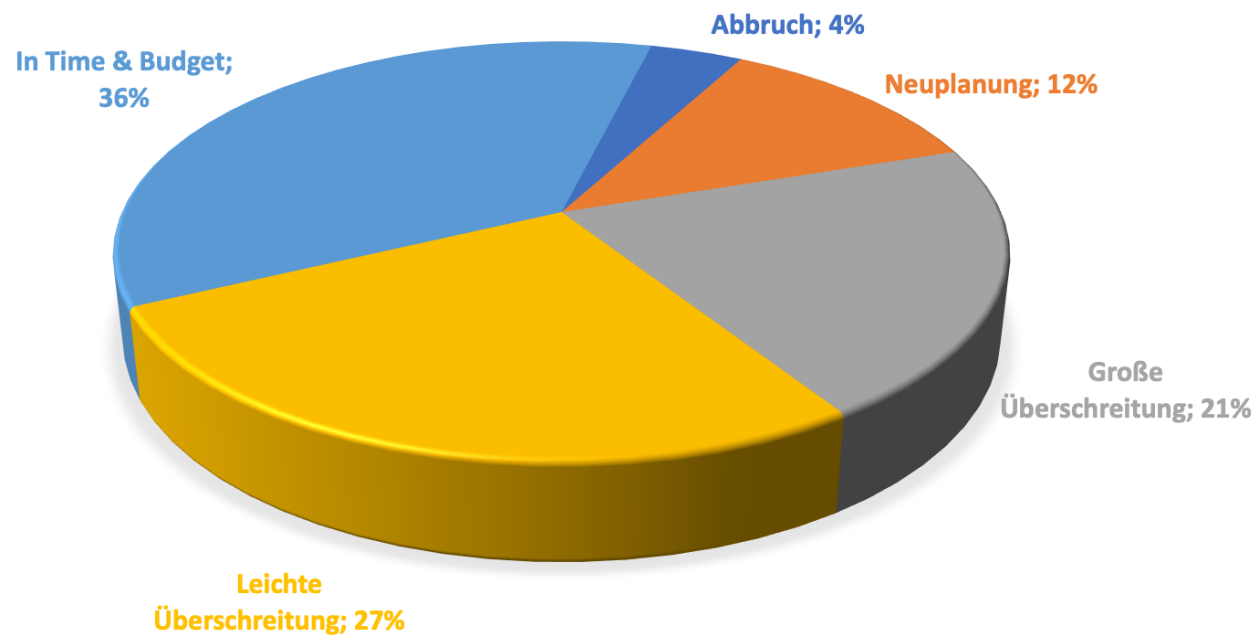
Software Engineering



Warum ist Software so teuer?

Warum ist Software so teuer?

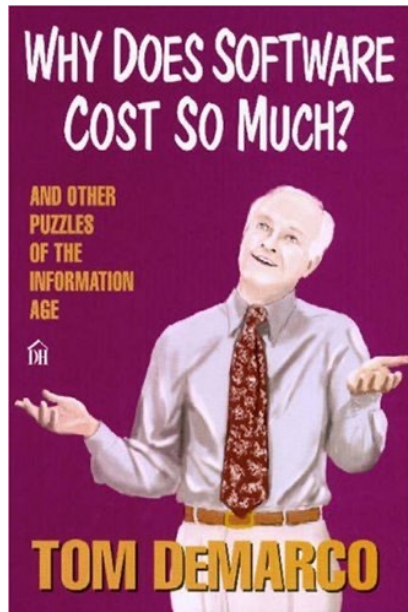
Verlauf von Software-Projekten



Quelle: swissQ,
"Trends & Benchmarks Report",
2014

Warum ist Software so teuer?

So lautet auch der (deutsche) Titel eines Buchs von Tom DeMarco



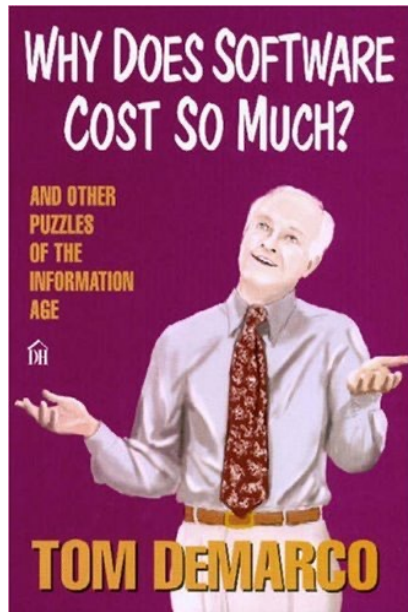
*"Haben Sie jemals einen dummen Fehler begangen?
Willkommen in der realen Welt."*

*"Haben Sie diesen Fehler hundertmal hintereinander gemacht?
Willkommen in der Software-Entwicklung."*

Tom DeMarco

Warum ist Software so teuer?

So lautet auch der (deutsche) Titel eines Buchs von Tom DeMarco



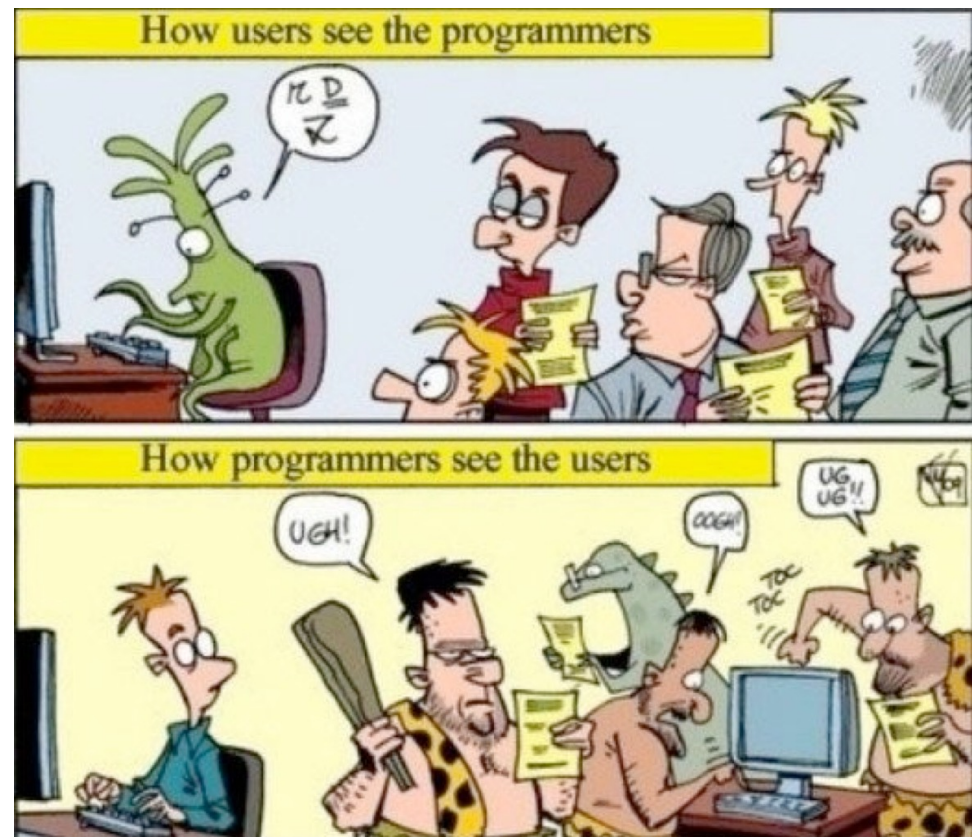
Kernaussage: Häufig ist nicht die Technik das Problem, sondern die Zusammenarbeit der Menschen, die die Lösung entwickeln sollen.

Warum ist Software so teuer?

Semantische Lücke

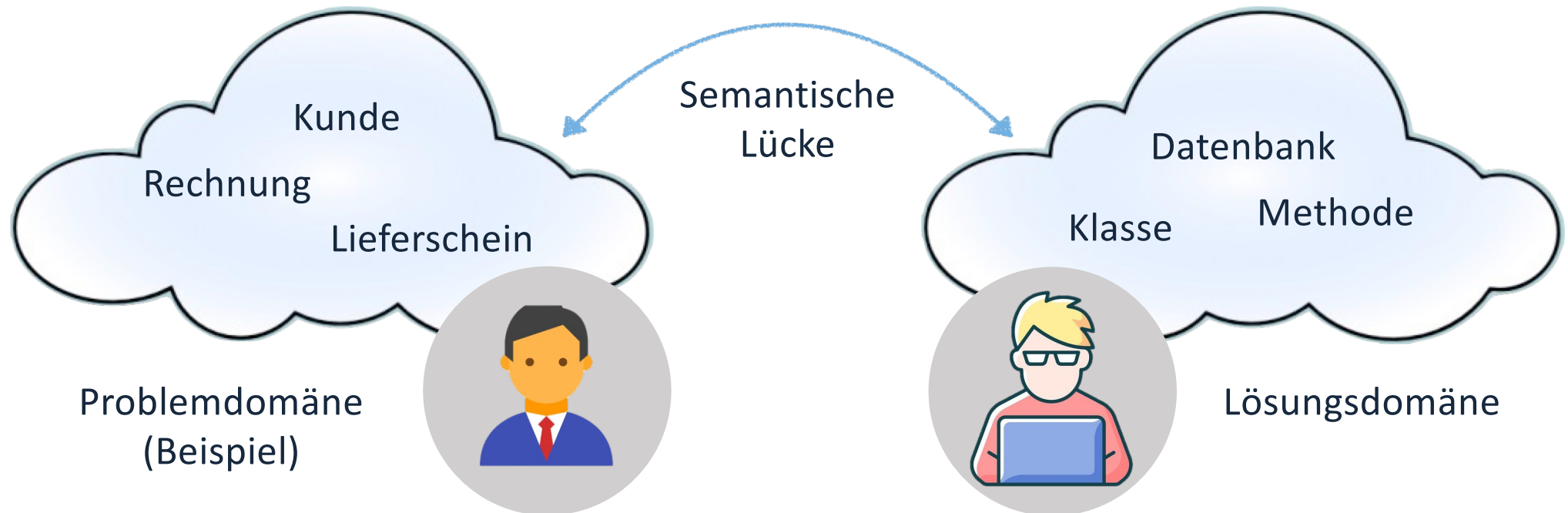
Oft sprechen Entwickler von Software nicht die Sprache derer, die die Software nutzen.

Umgekehrt haben die Nutzer Probleme, die Sprache der Entwickler zu verstehen.



Warum ist Software so teuer?

Semantische Lücke



Warum ist Software so teuer?

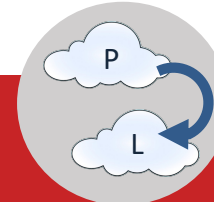
Die Herausforderung



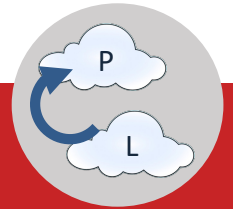
Das Problem (und auch die gewünschte Lösung) wird mit den Ausdrucksmitteln der Problemdomäne beschrieben.



Software wird konzipiert und entwickelt mit den Ausdrucksmitteln der Lösungsdomäne.



Die Aufgabenstellung und –lösung muss schrittweise von der Problemdomäne in die Lösungsdomäne fehlerarm überführt werden.



Umgekehrt müssen auch (Zwischen-) Ergebnisse der Lösungsdomäne in der Problemdomäne gefunden und überprüft werden.

Tätigkeiten

Tätigkeiten

Um die Herausforderung zu meistern, muss das Softwareentwicklungsteam eine Reihe von Tätigkeiten ausführen:

In dieser Vorlesung verwenden wir die rot umrandeten Begriffe. Abweichende Begriffe und Aufteilungen sind möglich.

Analyse

auch: **Requirements Engineering**

Erhebung und Klärung der Anforderungen an die Software zusammen mit den Anwendern

Design

auch: **Entwurf** oder **Architektur**

Technische Spezifikationen; Festlegung der zu Verwendenden Werkzeuge etc.

Implementierung

auch: **Programmierung** oder **Coding**

Erstellung ausführbarer bzw. masch. interpretierbarer Bausteine der Lösung

Integration

Zusammenführung getrennt entwickelter Komponenten zu einem lauffähigen Gesamtsystem

Roll Out

auch: **Deployment** oder **Produktivsetzung**

Bereitstellung der entwickelten Lösung für die Anwender

Tätigkeiten

Ordnen Sie zu!

Anlegen von Datenbanktabellen in der Oracle-Datenbank

Abstimmen des Aufbaus der Oberfläche mit den zukünftigen Anwendern

Festlegen des Technologie-Stacks (Editor, Programmiersprache, Datenbank)

Schulung der Anwender zu neuen Programmfeatures

Zusammenführen eines Feature-Branch mit dem Branch main

Analyse

Design

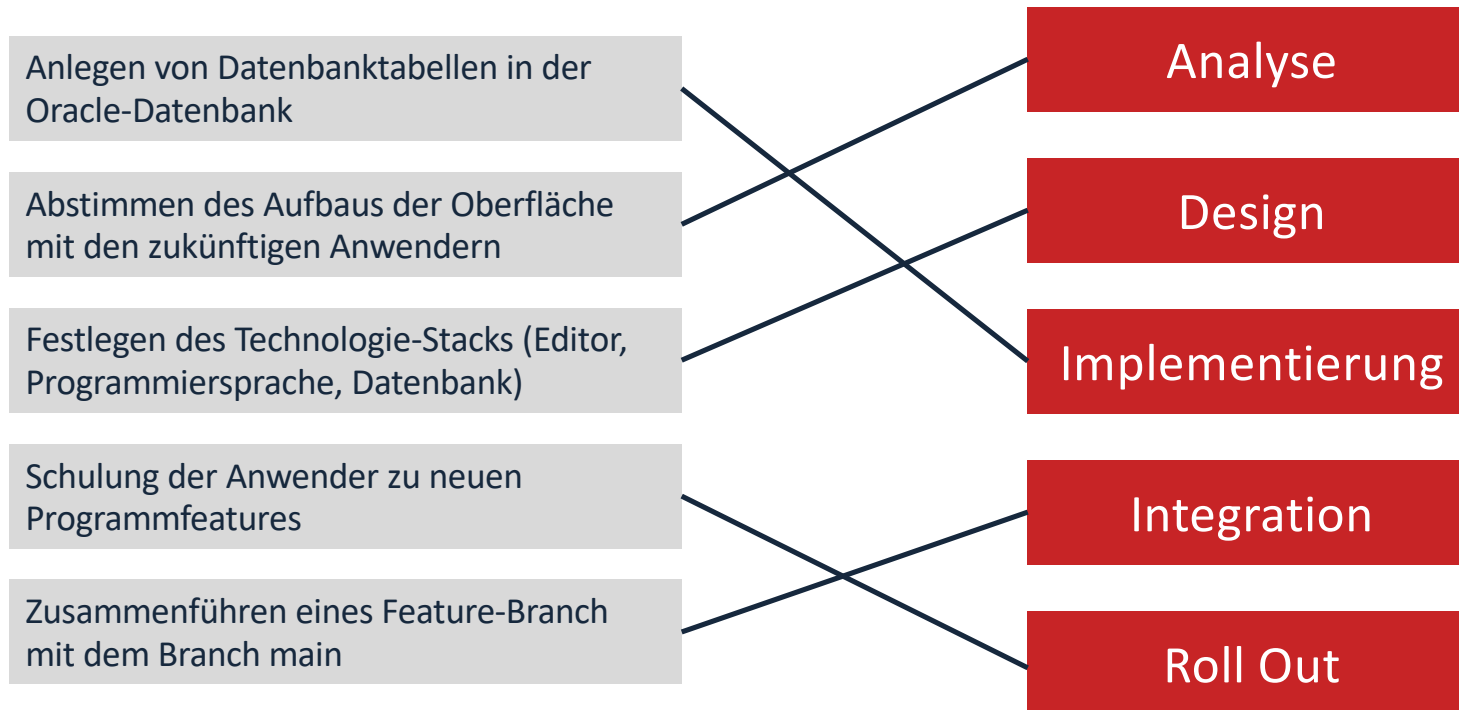
Implementierung

Integration

Roll Out

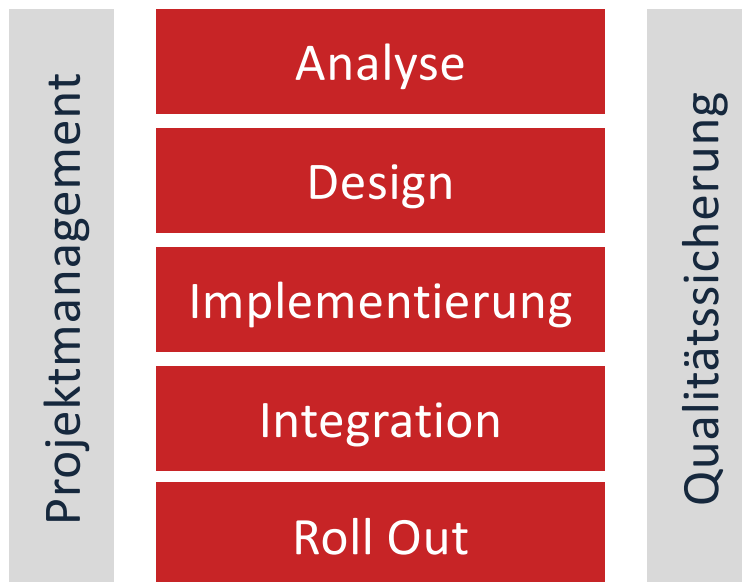
Tätigkeiten

Ordnen Sie zu!



Tätigkeiten

Organisation eines Software - Entwicklungsprojekts



In einem Software . Entwicklungsprojekt sind alle diese Tätigkeiten durchzuführen, flankiert von Querschnittsaufgaben wie **Projektmanagement** und **Qualitätssicherung**.

Es bleibt aber die Frage:

Wer macht wann was?

Klassische Vorgehensmodelle

Vorgehensmodelle

Festlegungen

- In welcher Reihenfolge werden Anforderungen umgesetzt?
- Wie werden die Tätigkeiten miteinander verzahnt?
- Wer ist für welche Tätigkeit verantwortlich?
- Wie funktioniert die Abstimmung untereinander?
- Wie ist bei Fehlern vorzugehen?
- Wer trifft Entscheidungen?
- Wie funktioniert das (kaufmännische) Controlling?

Vorgehensmodelle

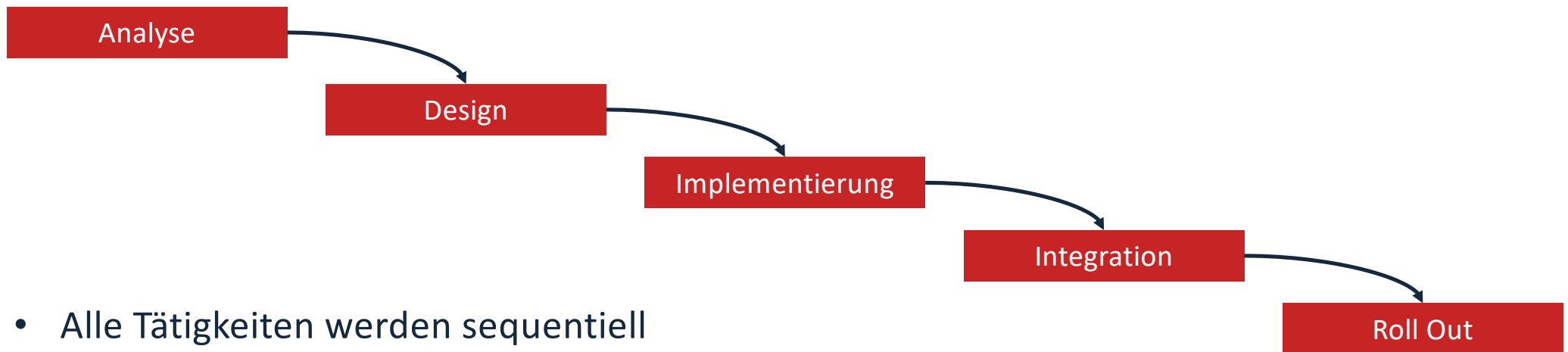
"Hey-Joe-Prinzip"



- Der "Chef" (Geschäftsführer, Projektleiter, ...) steuert das Team durch Tagesbefehle
- Anwender und Kunden wenden sich an den Chef, um Zugriff auf Teammitglieder zu bekommen
- Einsatz der Teammitglieder "wo am nötigsten"
- Tätigkeit (Analyse, Design, ...) wechselt je nach Tagesbefehl

Vorgehensmodelle

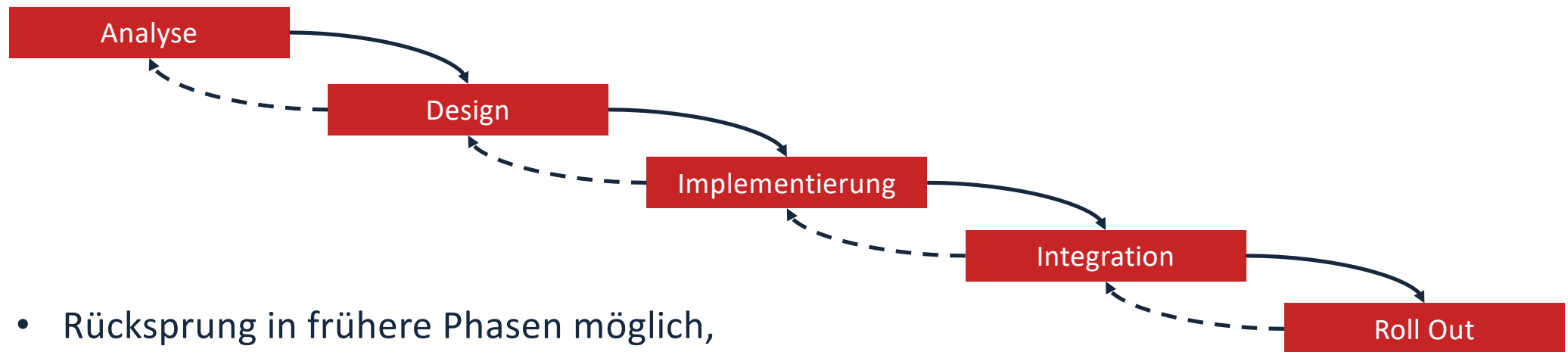
Wasserfallmodell



- Alle Tätigkeiten werden sequentiell in Phasen durchlaufen (Phasenmodell)

Vorgehensmodelle

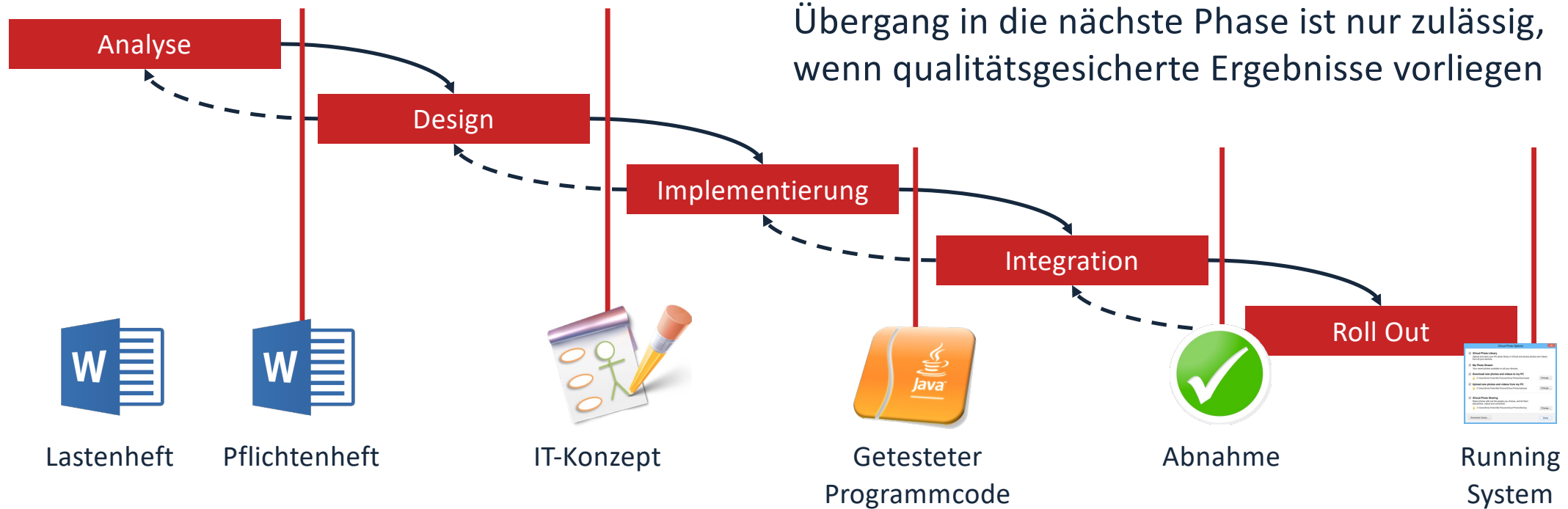
Wasserfallmodell – Verhalten bei Fehlern



- Rücksprung in frühere Phasen möglich, aber aufwendig
- "Spät erkannte Fehler sind teure Fehler"

Vorgehensmodelle

Wasserfallmodell – Meilensteine als "Quality Gate"



Vorgehensmodelle

Vergleich Wasserfallmodell vs. Hey Joe

Produktivität
des Teams



Flexibilität
bei Änderungen



Zufriedenheit
Anwender



Zufriedenheit
Mitarbeiter



Wasserfall

Hey Joe

Vorgehensmodelle

Vergleich Wasserfallmodell vs. Hey Joe

Produktivität
des Teams



Flexibilität
bei Änderungen



Zufriedenheit
Anwender



Zufriedenheit
Mitarbeiter



Wasserfall



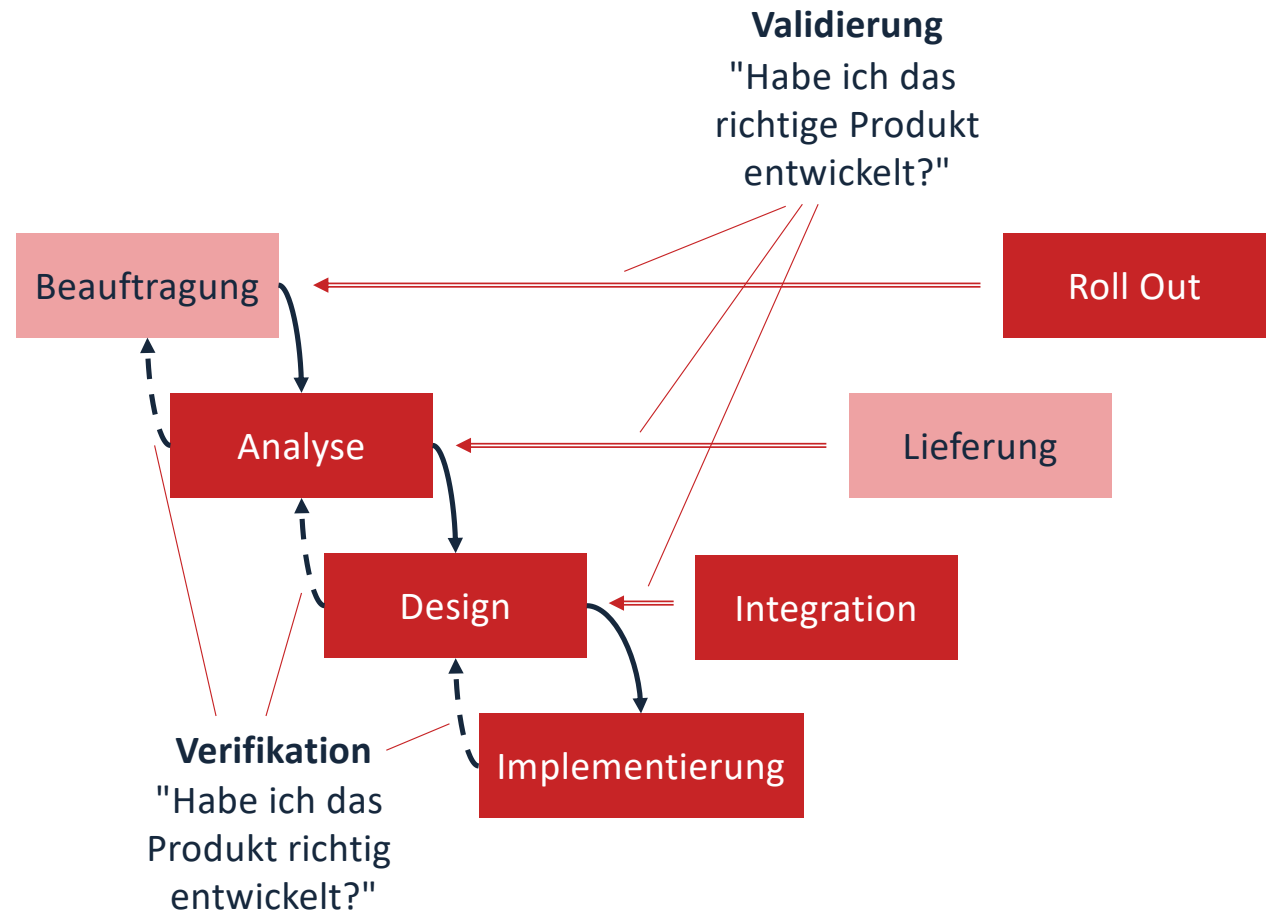
Hey Joe



Vorgehensmodelle

V-Modell

Das V-Modell ist eine Erweiterung des Wasserfallmodells. Es unterscheidet konstruktive Tätigkeiten, die immer weiter in Richtung der Lösungsdomäne verfeinern, und stellt diesen prüfende Tätigkeiten gegenüber. Während die konstruktiven Tätigkeiten wie beim Wasserfall jeweils eine **Verifikation** der Vorphase vornehmen, ergänzen die prüfenden Tätigkeiten eine **Validierung**.

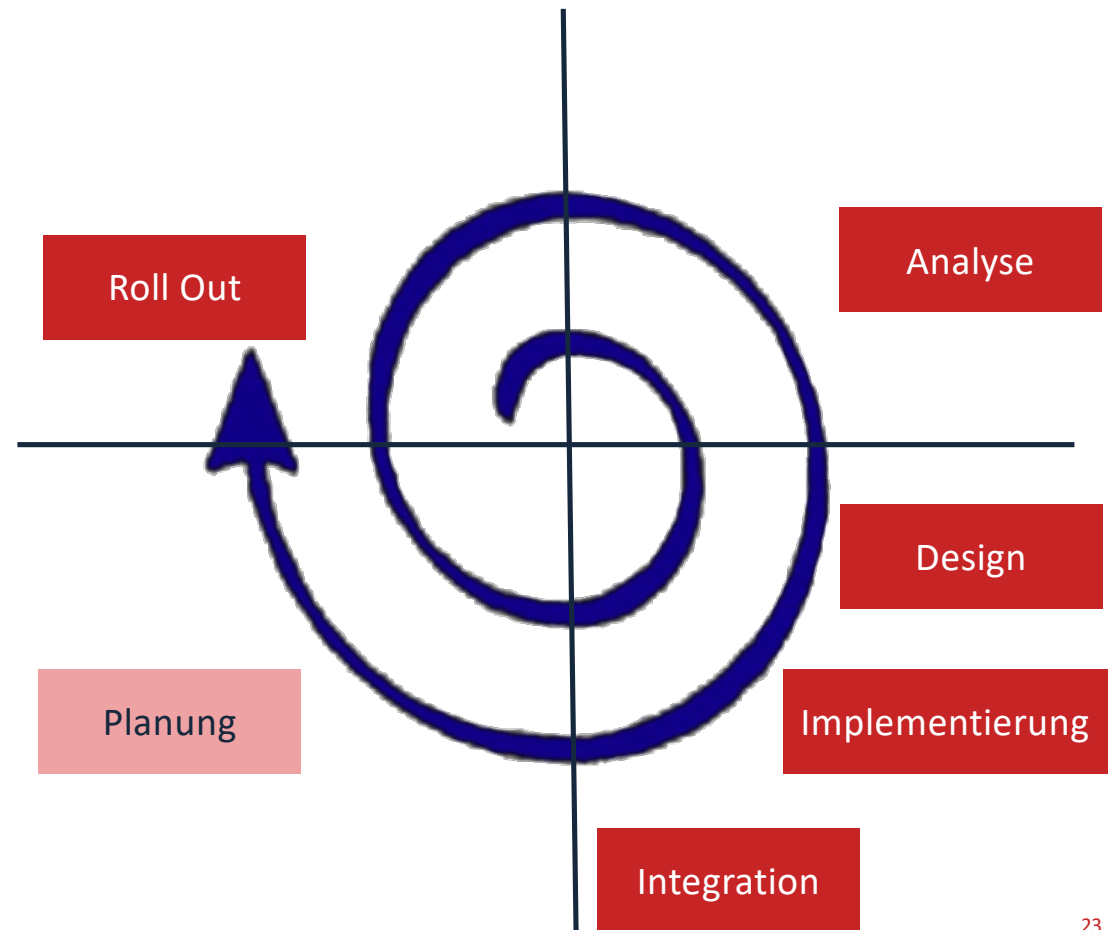


Vorgehensmodelle

Spiralmodell

Beim Spiralmodell wird die Entwicklung in einzelne **Iterationen** unterteilt, so dass nach jeder Iteration ein Zwischenprodukt zur Auslieferung steht. Die erste Iteration (Nullversion) startet mit dem Produktkern, in dem die ersten grundlegenden Anforderungen realisiert werden. Auf dieser Basis wird das Produkt durch zyklisches Durchlaufen der Phasen immer weiterentwickelt.

Das Grundproblem bei dieser Vorgehensweise ist jedoch, dass in späteren Iterationen Ergebnisse früherer Iterationen in Frage gestellt werden können. Dies kann zu **ineffizienten Doppelarbeiten** führen.



Lean Software Development

Lean Software Development

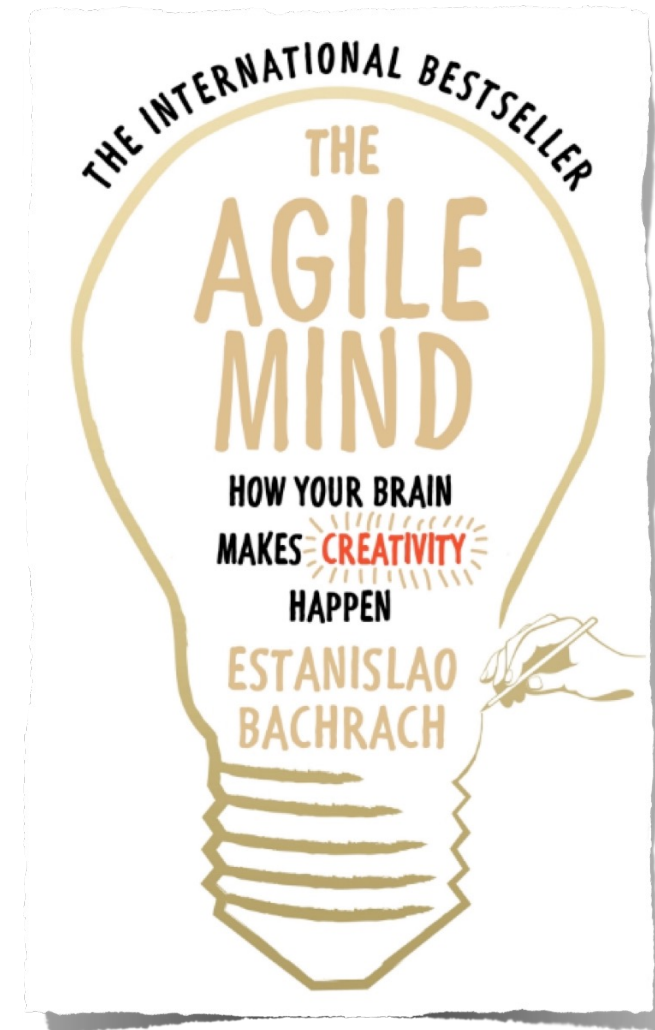
Studie

Das Experiment:

- sehr erfahrene Softwarearchitekten
- Design und Implementierung einer Fahrstuhlsteuerung
- in ca. 90 min lösbar

Die Fragestellung:

- verwenden die Experten einen Top-Down-Ansatz (z.B. Wasserfallmodell)?
- zerlegen sie das Problem schrittweise?
- setzen sie die Detaillösungen anschließend zusammen?

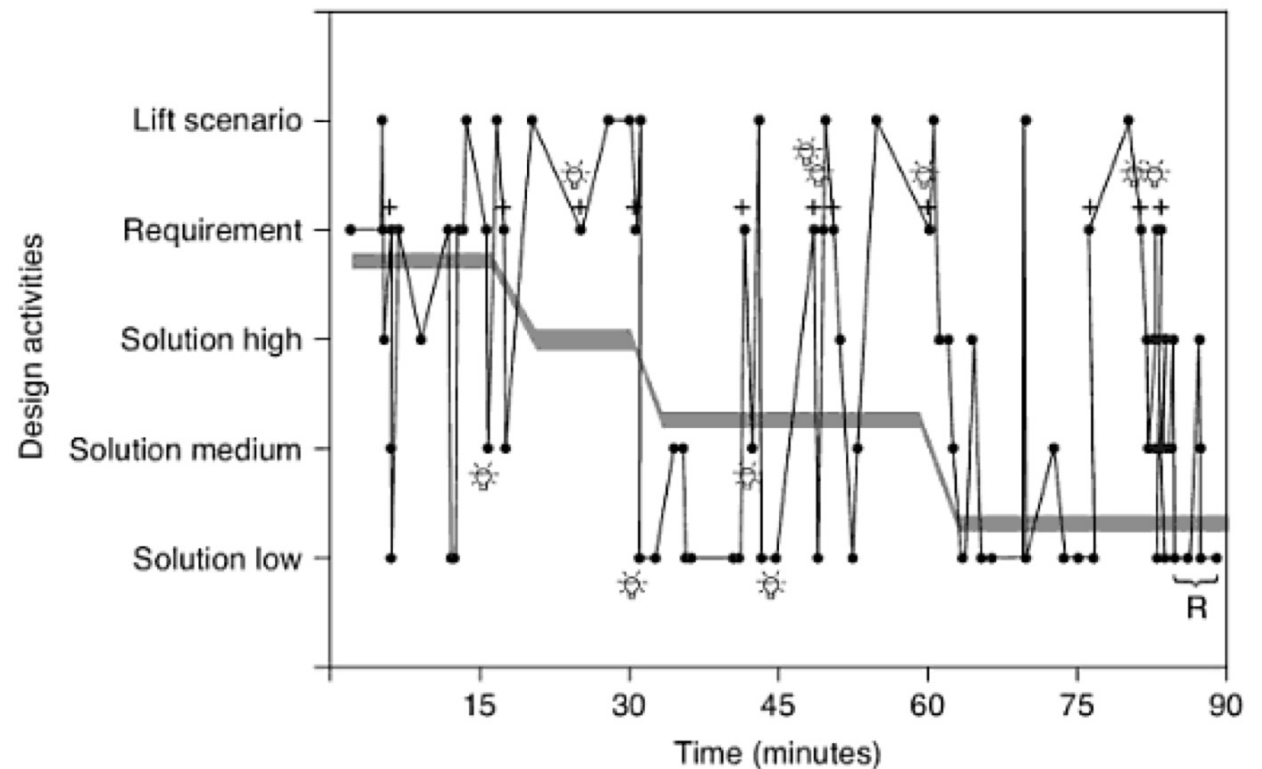


Lean Software Development

Studie

Das Ergebnis:

- Die erfahrenen Architekten wechseln rasch und häufig zwischen verschiedenen Abstraktionsebenen.
- Lösungsideen entstehen auf allen Ebenen und befördern offensichtlich den Ebenenwechsel.



Lean Software Development

Agiles Manifest (2001)

Agile Softwareentwicklung verfolgt den Ansatz, die Ideen des *Lean Thinking* auf die Erstellung von Software zu übertragen.

Wir erschließen bessere Wege, Software zu entwickeln,
indem wir es selbst tun und anderen dabei helfen.
Durch diese Tätigkeit haben wir diese Werte zu schätzen gelernt:

Individuen und Interaktionen mehr als *Prozesse und Werkzeuge*
Funktionierende Software mehr als *umfassende Dokumentation*
Zusammenarbeit mit dem Kunden mehr als *Vertragsverhandlung*
Reagieren auf Veränderung mehr als das *Befolgen eines Plans*

Das heißt, obwohl wir die Werte auf der rechten Seite wichtig finden,
schätzen wir die Werte auf der linken Seite höher ein.

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler

James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick

Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

Vielen Dank für die Aufmerksamkeit!

Fragen?